

Partner Portal Frontend Implementation Plan

Phased build plan + copy-paste prompts for each phase

Scope: Frontend only - React / Next.js + Tailwind CSS, with React Query and a small client-state store.

Source basis: WhiteDavid23 Multi-Tenant Partner Portal HLD/LLD v1.0 (6 pages).

Core business flow

Login -> Dashboard -> Leads -> Follow-up / Demo -> Admitted or Lost -> Admission -> Commission -> Payout

The prompts are deliberately scoped so a coding agent works on only one phase at a time instead of regenerating the whole frontend.

Project rules for every phase

These rules keep the frontend aligned with the handover document and prevent frontend code from taking over backend responsibilities.

- Frontend only. Do not change Express, MongoDB, Mongoose, Redis, middleware, commission rules, duplicate-detection rules, or other backend logic.
- Use React / Next.js + Tailwind CSS. Use React Query for server state. Use Zustand (or the existing Redux Toolkit setup if the project already uses it) only for small global client/UI state.
- Keep API access inside a dedicated service layer. React components must not talk to MongoDB or storage directly.
- Treat roles and permissions as UI behavior only. The frontend may hide unavailable actions, but the backend remains the security authority.
- Never hardcode tenant isolation in the UI. The backend is responsible for server-side partnerId scoping.
- Only integrate endpoints that actually exist. If the handover does not define an endpoint needed for a screen, create a typed service placeholder / mock adapter and clearly mark it TODO instead of inventing a backend contract.
- For every API-driven screen include loading, empty, success, validation, unauthorized (401), forbidden (403), duplicate (409 where relevant), and generic error states.
- Keep the UI responsive for desktop and mobile, keyboard accessible, and reusable. Avoid one huge page component.
- At the end of each phase, report: files created/changed, routes added, API contracts used, mock/TODO contracts, and a short manual test checklist.
- Do not start the next phase automatically. Stop after this phase is complete.

Phase map

Phase	Focus	Primary output	Backend dependency
1	Foundation	App structure, design system, API/query foundation	None / existing config
2	Auth + role shell	Login UI, protected shell, role/permission gates	Auth contract may need confirmation
3	Dashboard	Role-aware dashboard shells and data widgets	Dashboard endpoints not specified in HLD
4	Leads core	Lead list, pagination, create lead, duplicate handling	GET + POST /api/v1/leads defined
5	Lead workflow	Lead detail, status, follow-ups, activity UI	Specific detail/update APIs not defined
6	Admissions + commissions	Admission and commission views, payout approval UI	Payout PATCH defined; other APIs unspecified
7	Admin areas	Team, partners, notifications, reports, audit UI	Most APIs unspecified
8	Integration + QA	Error handling, responsive polish, permissions, tests	Uses all confirmed contracts

PHASE 1 - Foundation and Design System

Goal: Create a clean frontend foundation that later phases can extend without rewriting the project.

Deliverables for this phase

- Inspect the existing frontend first and preserve useful structure instead of regenerating the app.
- Create/normalize route, components, features, services, hooks, types, store and utility folders.
- Set up a single API client with base URL from environment variables.
- Set up React Query provider and query-key conventions.
- Create reusable UI primitives: Button, Input, Select, Textarea, Modal/Drawer, Card, Badge, Table shell, Pagination shell, Skeleton, EmptyState, ErrorState, Toast hook/interface.
- Create shared TypeScript domain types for User, Role, Permission, Lead summary, Admission summary, Commission summary and Notification summary, but keep unknown backend fields optional.
- Create a simple responsive visual system: spacing, typography, form states, table/mobile-card behavior.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 1 TASK - FOUNDATION ONLY

1. Inspect the existing frontend and summarize the current structure in 8-12 lines.
2. Refactor only where necessary to create a scalable feature-based structure.
3. Configure the API client using NEXT_PUBLIC_API_URL (or the existing equivalent). Add centralized response/error normalization.
4. Configure React Query at the app root.
5. Add the reusable primitives listed in the deliverables.
6. Add shared frontend domain types without pretending unspecified backend fields are guaranteed.
7. Create a small example page/component that proves the design primitives and query provider work, but do not

build business features yet.

8. Do not implement Login, Dashboard, Leads, Admissions, Commissions, Team, Partners, Reports or Notifications in this phase.

Acceptance criteria:

- App runs with no console errors.
- No business page was prematurely implemented.
- API calls have one reusable client entry point.
- Reusable components are responsive and accessible.
- Finish by listing files changed and a manual smoke-test checklist.

PHASE 2 - Authentication, App Shell and Role-Based Navigation

Goal: Build the login experience and the protected dashboard shell so each role sees the correct navigation and actions.

Deliverables for this phase

- Login page with validation, loading and backend-error feedback.
- Auth state model for current user, role, partnerId and permissions.
- Protected route/app-shell behavior.
- Responsive sidebar/header/navigation.
- Reusable RoleGate / PermissionGate UI components.
- 401 behavior that clears/refreshes auth according to the available backend contract; 403 UI for denied actions.
- Navigation variants for Super Admin, Partner Admin, Team Member and Support, while only showing modules actually supported by requirements.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.

- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 2 TASK - AUTH + APP SHELL ONLY

Build authentication UI and the reusable protected application shell.

Requirements:

1. Create /login with email and password inputs, client-side validation and accessible errors.
2. Inspect the actual backend/codebase for the authentication endpoint and token/cookie behavior. If it is not available, create authService.login() as TODO-CONTRACT plus a development mock. Do not invent a production endpoint.
3. Model the current user as { id, role, partnerId, permissions } because the documented backend attaches these concepts to the authenticated request.
4. Add route protection and a clean sign-out path.
5. Add Sidebar + Header + MobileNavigation.
6. Implement PermissionGate and RoleGate for UI visibility.
7. Partner Admin and Team Member must not be given a UI control to choose another partner/tenant. Super Admin may later receive global partner controls.
8. Handle 401 as unauthenticated and 403 as authenticated-but-not-allowed.
9. Build placeholder routes for future modules, but do not implement their contents yet.

Stop after the shell is working. Report any missing auth contract instead of guessing it.

PHASE 3 - Role-Aware Dashboard

Goal: Create dashboard experiences that reflect the role and business flow without inventing unsupported business calculations.

Deliverables for this phase

- Reusable KPI card, pipeline summary, upcoming follow-ups list and recent-activity widgets.
- Partner Admin dashboard focused on own-partner leads, follow-ups, admissions and commission summaries.
- Team Member dashboard focused on assigned leads and follow-ups.
- Super Admin dashboard shell for global partner/lead/admission/payout summaries.
- Skeleton, empty and error states.
- Mock/dashboard adapter if aggregate endpoints do not exist.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 3 TASK - DASHBOARD ONLY

Build role-aware dashboard screens using reusable widgets.

Use the documented business concepts only: leads move through new -> contacted -> follow_up -> demo -> admitted/lost; admissions come from converted leads; commissions have earned/pending/paid concepts.

1. Create a dashboard route and reusable widgets: KpiCard, PipelineSummary, UpcomingFollowUps, RecentActivity, CommissionSummary.
2. Partner Admin: show own-partner summary UI.
3. Team Member: show assigned-work summary UI.
4. Super Admin: show global summary UI.
5. Support: create a minimal neutral dashboard shell unless the codebase/spec provides stronger requirements.
6. Search the existing frontend/backend contracts for real dashboard endpoints. If none exist, build a dashboardService interface with a mock adapter and mark all unresolved methods TODO-CONTRACT. Do not derive totals by downloading huge datasets.
7. Do not implement lead management actions, admission forms, payout approval or admin CRUD in this phase.
8. Make dashboard widgets responsive and usable on narrow screens.

Finish with a screenshot-oriented manual test checklist for each role.

PHASE 4 - Leads Core - List, Pagination and Create

Goal: Implement the most important CRM feature using the API contracts explicitly defined in the handover.

Deliverables for this phase

- Leads route with list/table and mobile card layout.
- Pagination using response meta.page, meta.limit and meta.total.

- Search/filter UI that only sends parameters supported by the real backend.
- Create Lead modal/page with fields supported by the documented request body.
- Display lead status, priority and leadScore from backend data.
- 409 duplicate lead feedback with a human-readable message.
- 401/403/400/general error handling and success toast/refetch.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 4 TASK - LEADS CORE ONLY

Use these confirmed contracts from the HLD/LLD:

- GET /api/v1/leads returns { success, data: [...], meta: { page, limit, total } }.
- POST /api/v1/leads accepts at least studentName, phone, email, courseId, source, priority, budget.
- Successful create returns data including _id, partnerId, status, leadScore, isDuplicate, createdAt.
- Duplicate may return HTTP 409 with error.code = DUPLICATE_LEAD.

Build:

1. /leads list page with responsive table/cards.
2. Pagination controlled by server meta, not fake client slicing.
3. Filters/search UI only for parameters confirmed by the existing backend. If a parameter is not supported, keep it client-disabled/TODO instead of inventing it.
4. Add Lead flow using a modal or dedicated route.
5. Client validation for required fields and friendly field messages.
6. On POST success: show success feedback, close/reset form and invalidate/refetch the leads query.
7. On 409: show 'This phone number already exists as a lead for this partner' or the backend message.
8. On 400: map backend validation errors to fields where possible.
9. Do not let Partner Admin/Team Member choose or override partnerId in this form. Tenant scoping is a backend concern.
10. Do not calculate leadScore or duplicate status in the frontend. Display backend values only.

Do not build lead details/follow-up editing yet.

PHASE 5 - Lead Detail and Follow-Up Workflow

Goal: Build the detailed lead workspace and visualize the documented lead progression, while keeping unspecified update/activity contracts isolated.

Deliverables for this phase

- Lead detail page shell for student/contact/course/source/priority/status/leadScore/assignment/followUpDate.
- Status badge/stepper for new, contacted, follow_up, demo, admitted, lost.
- Follow-up section and activity/timeline UI.
- Assigned-to presentation and permission-aware action buttons.
- Service interfaces for detail/update/follow-up/activity contracts if they are not already implemented by the backend.
- No frontend calculation of lead scoring or duplicate rules.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 5 TASK - LEAD DETAIL + WORKFLOW ONLY

The documented Lead fields include partnerId, courseId, assignedTo, studentName, phone, whatsapp, email, city, source, priority, status, budget, leadScore, followUpDate, isDuplicate and createdAt. The documented status enum is new | contacted | follow_up | demo | admitted | lost.

1. Create /leads/[id] (or the project routing equivalent) with a clean lead profile layout.
2. Add a status visualizer for the documented pipeline. Treat admitted and lost as terminal outcomes.
3. Add sections for next follow-up, assigned user and activity/timeline.
4. Inspect the actual API/codebase for GET-one, PATCH/update, assignment, follow-up and LeadActivity endpoints.

5. Integrate only real contracts. For anything missing, create typed service methods behind TODO-CONTRACT mocks so the UI can be reviewed without pretending a backend API exists.
6. Permission-gate update controls. A Team Member should primarily act on assigned records according to the documented role model.
7. Never calculate leadScore, determine duplicate status, or force tenant partnerId in the frontend.
8. Add unsaved-change protection if an edit form is present.

Do not implement admissions or commission screens in this phase.

PHASE 6 - Admissions and Commissions

Goal: Implement frontend views for converted leads, commission records and the confirmed Super Admin payout approval action.

Deliverables for this phase

- Admissions list/detail shells linked conceptually to converted leads.
- Commission list/detail with earnedAmount, pendingAmount, paidAmount and payout status.
- Partner Admin commission read view.
- Super Admin-only Approve Payout control using the documented PATCH endpoint.
- 403 handling if an unauthorized role attempts the action.
- No frontend commission calculations.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 6 TASK - ADMISSIONS + COMMISSIONS ONLY

Source behavior:

- An Admission is created when a lead converts and drives commission.
- CommissionRecord tracks earnedAmount, pendingAmount, paidAmount and payout lifecycle.
- Confirmed payout action: PATCH /api/v1/commissions/{id}/payout with { action: 'approve' }. This is Super Admin only and requires commission:approve_payout. A successful response includes payoutStatus and payoutDate; unauthorized roles may receive 403 FORBIDDEN.

Build:

1. Admissions list/detail UI using actual APIs if present; otherwise use typed TODO-CONTRACT adapters.
2. Commission list/detail UI that displays backend-provided monetary values; never calculate commission amounts in the browser.
3. Partner Admin: view own-partner commission information; no payout-approval button.
4. Super Admin: permission-gated Approve Payout action. Ask for confirmation before calling the PATCH endpoint.
5. On success, update/invalidate relevant commission queries and show a success toast.
6. On 403, show a permission error and keep the UI consistent.
7. Format currency/date for display only; do not alter monetary business logic.

Do not build Partner/Team management, Reports or Audit Logs yet.

PHASE 7 - Partners, Team, Notifications, Reports and Audit UI

Goal: Build the remaining role-specific administration areas as modular frontend features without inventing server contracts.

Deliverables for this phase

- Super Admin Partners list/detail/approval shell.
- Partner Admin Team management shell.
- Notifications center and unread indicator shell.
- Reports page shell for role-appropriate reporting.
- Super Admin Audit Log read-only screen.
- Support-role navigation/actions only where source/codebase confirms them.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC

middleware, tenant scoping, commission calculations, or duplicate detection.

- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 7 TASK - REMAINING ADMIN MODULES ONLY

The HLD defines these concepts/permissions but does not provide detailed REST contracts for most of them:

- Super Admin: global scope, partner approval, role management, report:view_all, audit:view.
- Partner Admin: own partner, lead/admission access, commission:view, team:manage.
- Partner has many Users and Notifications.
- AuditLog is write-once and Super Admin has read-only access.

1. Inspect the codebase for real Partner, User/Team, Notification, Report and Audit endpoints before coding data integration.
2. Build feature modules and polished UI shells for each area.
3. Integrate only confirmed endpoints. Put missing methods behind TODO-CONTRACT service interfaces and mock adapters.
4. Super Admin can receive partner-management UI; Partner Admin must not receive global tenant controls.
5. Audit Log UI is read-only. Do not add edit/delete actions.
6. Notifications should support a list and read/unread presentation if supported by the backend; otherwise keep that interaction mocked and clearly marked.
7. Reports must present backend data; do not invent financial or conversion formulas in the browser.
8. Keep each module separate so it could later be extracted or rewritten without affecting Leads.

Do not change backend routes or schemas to make the frontend easier.

PHASE 8 - Integration Hardening, Responsive QA and Handoff

Goal: Turn the phased frontend into one reliable, consistent product ready for backend integration/UAT.

Deliverables for this phase

- Centralized API error mapping for 400, 401, 403, 409 and generic failures.
- Consistent loading/skeleton/empty/error states across all modules.
- Responsive QA for mobile, tablet and desktop.
- Permission/role audit of every navigation item and action.
- Query invalidation/caching review and removal of duplicate fetches.

- Accessibility pass for labels, keyboard operation, focus management and contrast.
- Smoke tests for critical frontend flows.
- Final API contract/TODO-CONTRACT inventory for backend team handoff.

Copy-paste prompt

You are implementing ONLY the frontend of the WhiteDavid23 Multi-Tenant Partner Portal.

Global constraints:

- Stack: React / Next.js + Tailwind CSS.
- Use React Query for server-state fetching/caching. Use Zustand for small global UI/auth state unless the existing codebase already uses Redux Toolkit, in which case keep Redux Toolkit.
- Do not modify backend code, database schemas, MongoDB, Mongoose, Redis, auth middleware, RBAC middleware, tenant scoping, commission calculations, or duplicate detection.
- Do not invent backend endpoints. If a required endpoint is not documented or not present in the codebase, create a frontend service interface/mock and mark it TODO-CONTRACT.
- All API calls belong in a services/api layer, not directly inside page components.
- Build reusable components and keep pages thin.
- Include loading, empty, error and success states.
- Respect role/permission-based UI, while assuming the backend is the final security authority.
- Do not automatically continue into another phase.

PHASE 8 TASK - INTEGRATION + QA ONLY

Do not add new product modules. Harden what already exists.

1. Audit all routes for role/permission visibility.
2. Audit all API calls so they go through the shared service layer and React Query where appropriate.
3. Normalize error handling:
 - 400 VALIDATION_ERROR -> friendly field/general validation message
 - 401 UNAUTHENTICATED -> refresh/re-auth flow according to the real auth contract, otherwise redirect to login
 - 403 FORBIDDEN -> permission message
 - 409 DUPLICATE_LEAD -> human-readable duplicate warning
 - network/5xx -> retry-safe generic message
4. Ensure every data screen has loading, skeleton, empty and error states.
5. Check mobile/tablet/desktop layouts, especially tables, forms, modals, sidebar and dashboards.
6. Audit query keys, stale times and invalidation after mutations; remove unnecessary duplicate requests.
7. Check accessibility: semantic labels, keyboard navigation, focus trapping in modal/drawer, visible focus, useful button names.
8. Add smoke tests for: login/shell, role navigation, lead list pagination, lead creation success, duplicate 409, permission 403, commission payout approval for Super Admin.
9. Produce FINAL_FRONTEND_HANDOFF.md containing:
 - routes implemented
 - components/features implemented
 - confirmed backend endpoints used
 - every TODO-CONTRACT and what request/response shape the frontend needs
 - environment variables
 - manual QA checklist

Appendix A - Confirmed API contracts from the handover

Create Lead

- POST /api/v1/leads
- Request body includes: studentName, phone, email, courseId, source, priority, budget.
- 201 response includes: _id, partnerId, status, leadScore, isDuplicate, createdAt.
- 409 response may use error.code = DUPLICATE_LEAD.

List Leads

- GET /api/v1/leads
- 200 response: { success, data: [...], meta: { page, limit, total } }.
- Partner data is tenant-scoped by backend middleware before controller/business logic.

Approve Commission Payout

- PATCH /api/v1/commissions/{id}/payout
- Request: { action: "approve" }.
- Super Admin only; permission: commission:approve_payout.
- 200 returns payoutStatus/payoutDate; 403 is FORBIDDEN when role lacks permission.

Common errors in lead-create flow

- 401 UNAUTHENTICATED - missing/expired JWT.
- 403 FORBIDDEN - role lacks lead:create.
- 400 VALIDATION_ERROR - invalid phone/email/etc.
- 409 DUPLICATE_LEAD - duplicate partnerId + phone.

Appendix B - Source-derived frontend boundaries

- The architecture is multi-tenant by partnerId; Super Admin has cross-tenant visibility while partner users are scoped to their own tenant.
- Server-side tenant scoping is a design goal; it must not rely on hiding data in the UI.
- The documented frontend technology is React (Next.js) + Tailwind CSS, with Redux Toolkit / Zustand and React Query.
- The backend is Node.js + Express.js, data is MongoDB Atlas via Mongoose, cache is Redis, and file storage is S3/Cloudinary. The frontend should consume REST APIs rather than access these data services directly.

- The lead workflow is Lead Created -> Duplicate Check -> Assigned & Scored -> Follow-up / Demo -> Admitted or Lost.
- The lead status enum is new | contacted | follow_up | demo | admitted | lost.

Recommended working method

- Run one phase prompt at a time.
- Review the changed files and UI before starting the next phase.
- If the coding agent finds an undocumented API need, add it to the TODO-CONTRACT list instead of allowing it to guess.
- Keep mock data behind the same service interfaces that production APIs will later use, so replacing mocks does not require rewriting components.
- After Phase 8, use FINAL_FRONTEND_HANDOFF.md as the integration checklist with the backend developer.